AWS Lab: Security Groups and Network ACLs

VPC and EC2 Service

29th October 2026

1 Objective

The primary objective of this experiment is to design, implement, and validate a secure, multi-tier network architecture within an AWS Virtual Private Cloud (VPC). By provisioning isolated public and private subnets, this practical exercise aims to demonstrate the "Defense in Depth" security model.

Specifically, the experiment focuses on achieving the following key goals:

- **VPC and Subnet Isolation:** To configure a custom VPC and partition it into distinct routing zones (Public and Private subnets) utilizing custom Route Tables and an Internet Gateway.
- **Stateful Instance-Level Security:** To deploy Amazon EC2 instances and enforce fine-grained access control using Security Groups, highlighting their stateful behavior.
- **Stateless Subnet-Level Security:** To implement custom Network Access Control Lists (NACLs) as an explicit boundary firewall, demonstrating their stateless requirement for defining both inbound and outbound traffic rules.
- **Bastion Host Architecture:** To securely communicate with a private, non-routable backend server by routing SSH and ICMP traffic exclusively through a hardened public-facing instance.

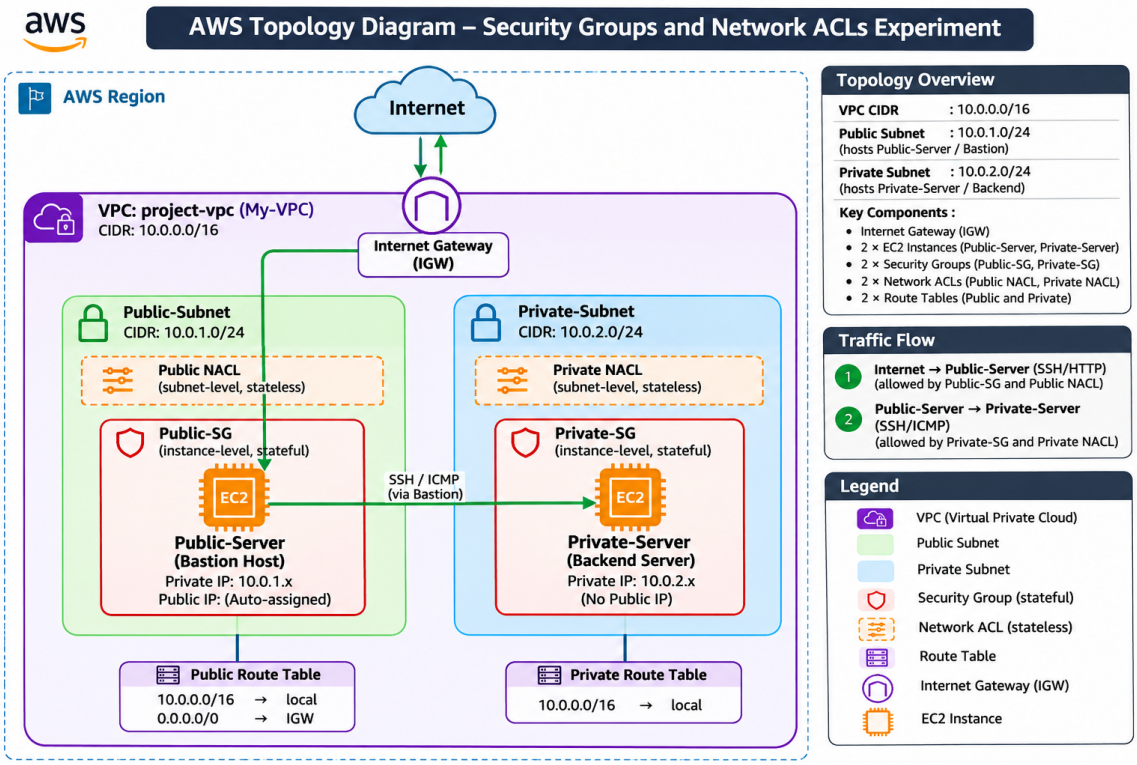


Figure 1: Network topology for the AWS VPC experiment

2 VPC and Subnet Configuration

2.1 Creating the Custom VPC

The first step involves creating a custom VPC to serve as the foundational isolated network environment. A VPC named **project-vpc** (or **My-VPC**) was provisioned with an IPv4 CIDR block of `10.0.0.0/16`. Both DNS resolution and DNS hostnames were enabled for this network space to ensure instances can resolve domain names appropriately.

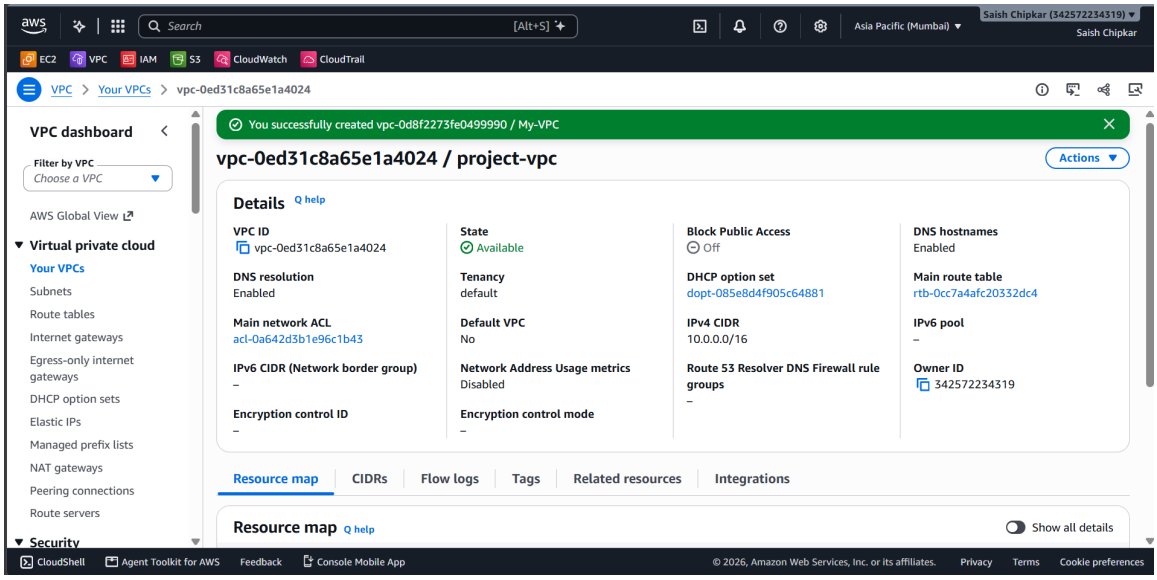


Figure 2: VPC Dashboard showing the created VPC details

2.2 Subnet Allocation

To separate resources based on their required accessibility, the VPC address space was divided into two distinct subnets:

- **Public-Subnet**: Designed to host resources that require direct inbound and outbound access to the public internet (e.g., bastion hosts, web servers).
- **Private-Subnet**: Designed to host internal resources that should be entirely isolated from direct internet access (e.g., databases, backend application servers).

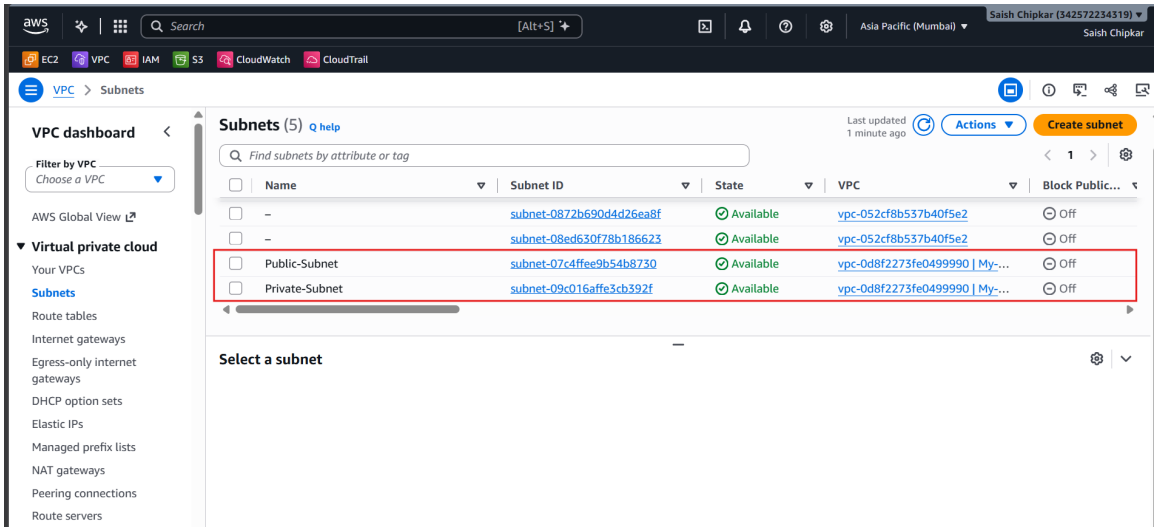


Figure 3: Subnets dashboard displaying Public-Subnet and Private-Subnet

3 Provisioning EC2 Instances

With the network foundation established, two Amazon EC2 instances were launched—one in each subnet—to practically test the firewall rules.

3.1 Public Server Deployment

The launch wizard was initiated to provision the first instance, assigned the Name tag **Public-Server**.

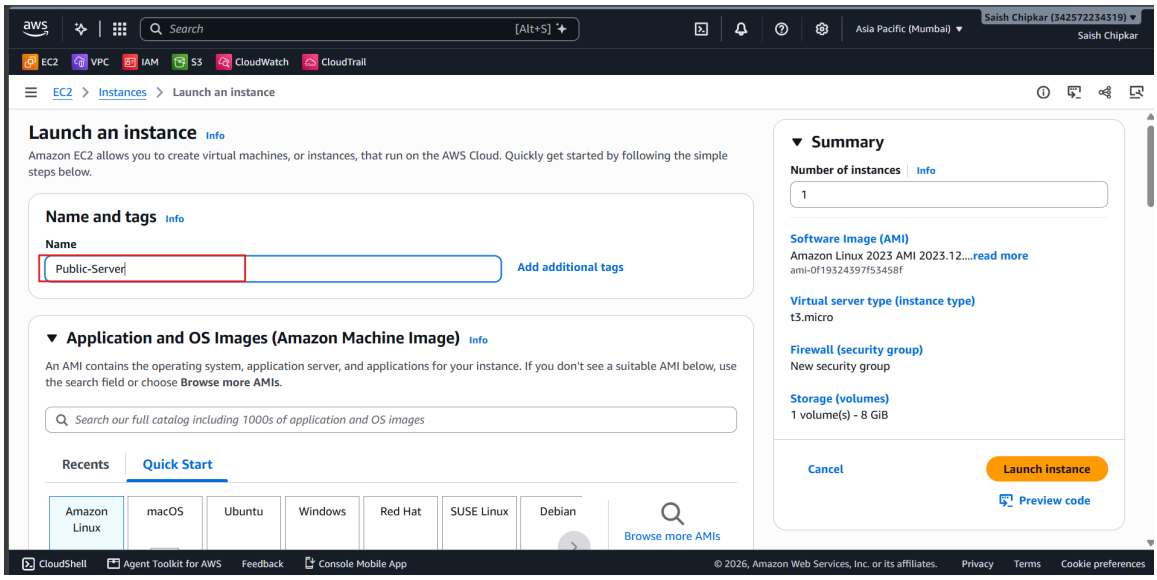


Figure 4: Launch Instance wizard showing the name 'Public-Server'

In the Network Settings configuration pane, this instance was intentionally placed within the **My-VPC** network and associated with the **Public-Subnet**. To ensure the instance is reachable over the internet, the **Auto-assign public IP** option was set to `Enable`.

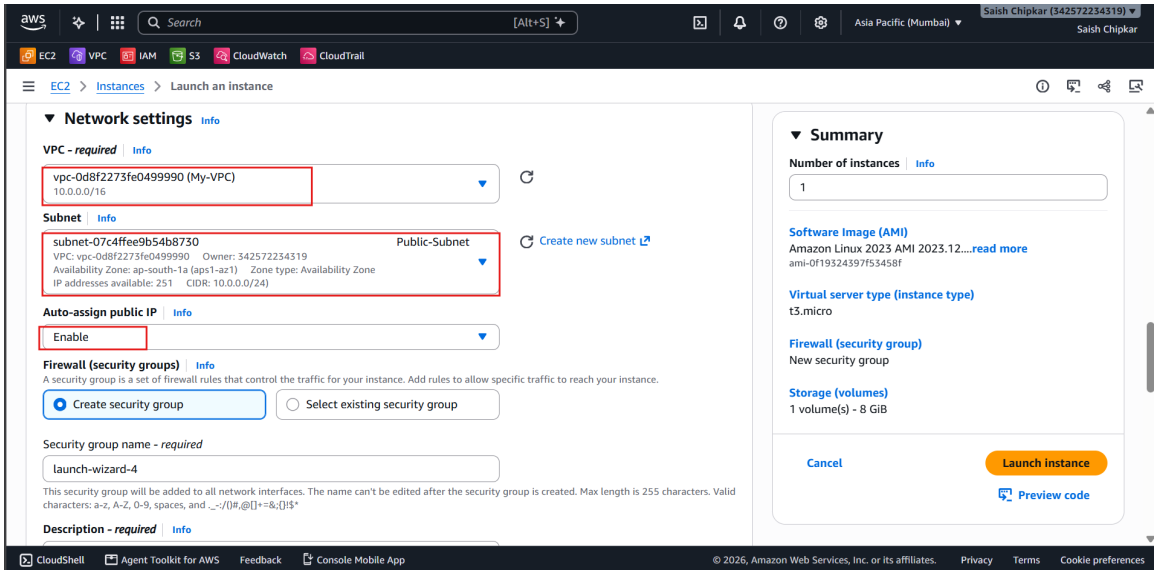


Figure 5: Network settings mapping to Public-Subnet with public IP enabled

A dedicated security group named **Public-SG** (with the description "public-server security group") was created. This Security Group will later be configured with inbound rules to permit necessary public traffic (such as SSH or HTTP).

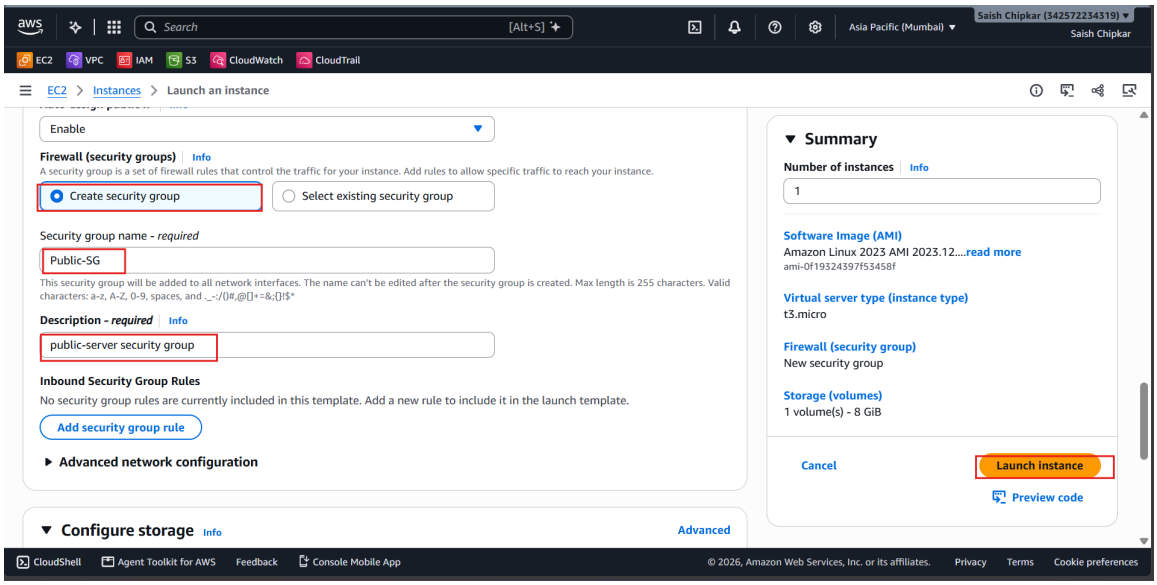


Figure 6: Security group creation for Public-SG

3.2 Private Server Deployment

A second instance was launched and assigned the Name tag **Private-Server**. This instance simulates a backend resource.

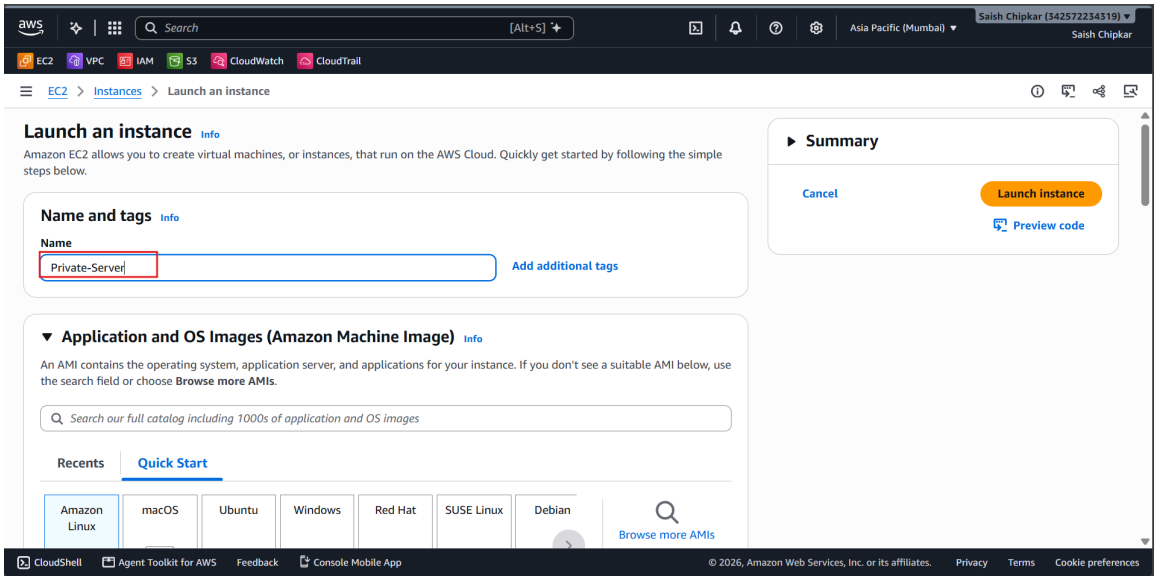


Figure 7: Launch Instance wizard showing the name 'Private-Server'

In its Network Settings, the instance was assigned to the **Private-Subnet**. Crucially, the **Auto-assign public IP** option was set to `Disable`. This ensures the server does not receive a public routable IP address, enforcing its isolation from the internet.

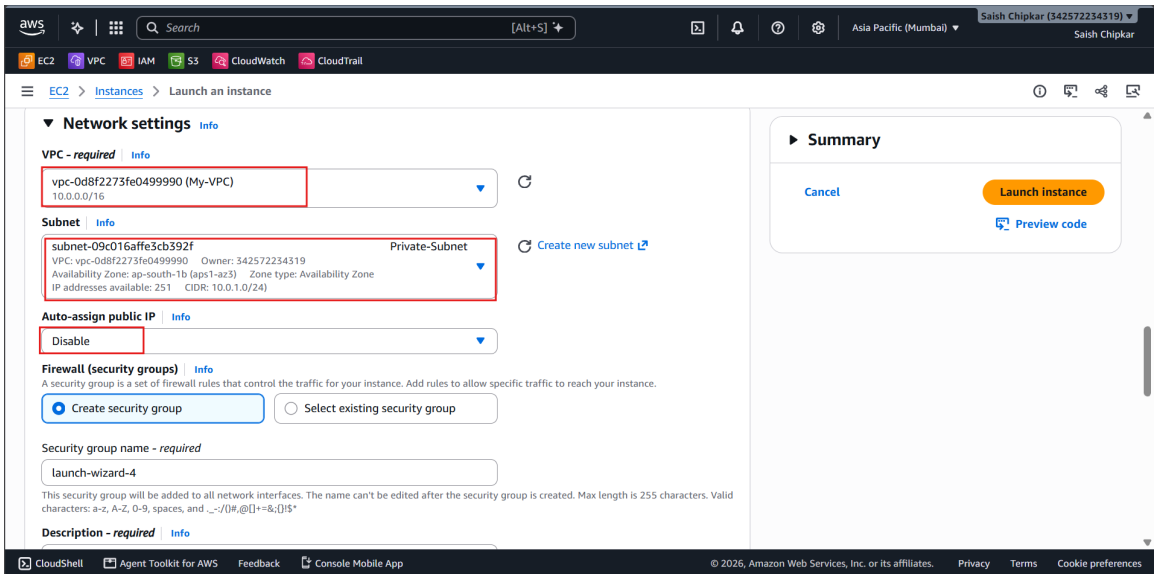


Figure 8: Network settings mapping to Private-Subnet with public IP disabled

A separate security group named **Private-SG** ("Private-Server Security Group") was created for this instance. This group will strictly control internal traffic, ensuring only authorized instances (like the Public-Server) can communicate with it.

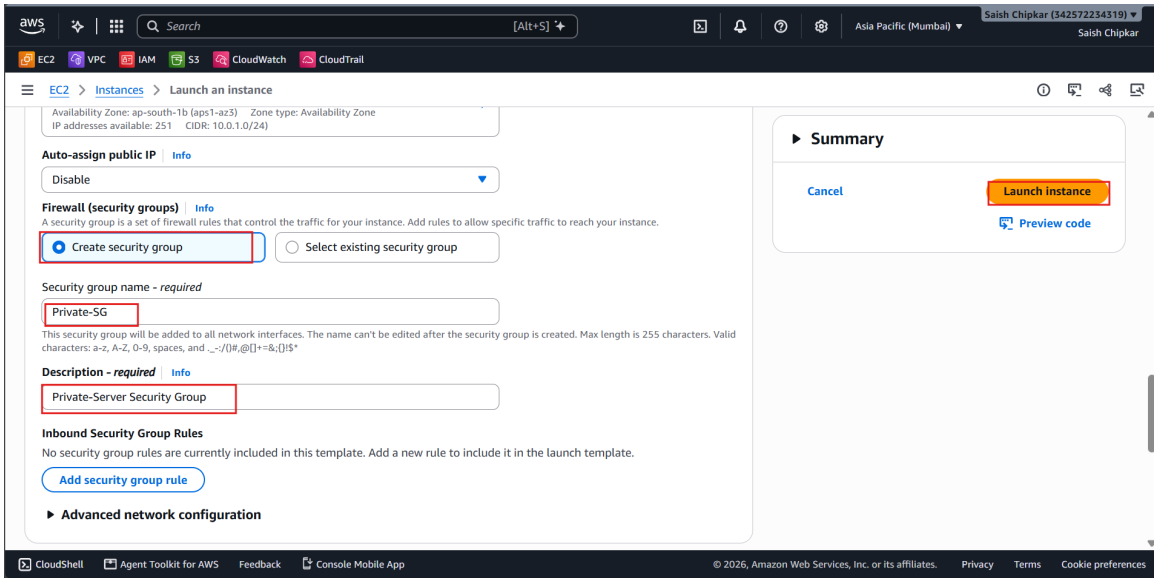


Figure 9: Security group creation for Private-SG

4 Network ACL Initialization

After configuring instance-level Security Groups, the focus shifts to subnet-level protection using Network ACLs (NACLs). By default, the custom VPC comes with a default NACL that allows all inbound and outbound traffic, which is currently associated with both the Public and Private subnets.

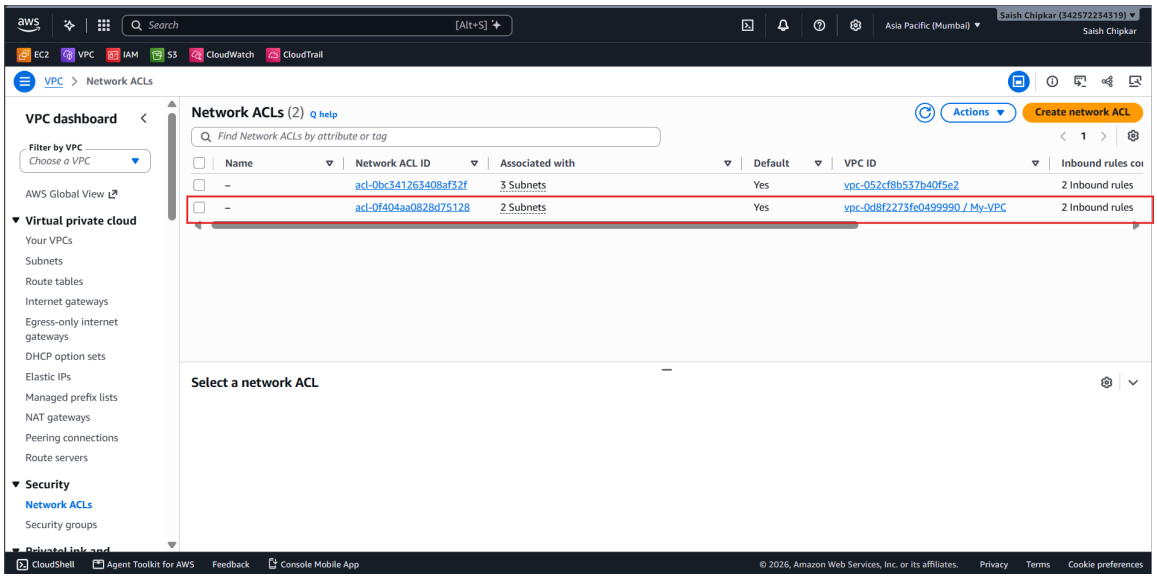


Figure 10: Network ACLs dashboard showing the default permissive ACL

To enforce strict, explicit network boundaries and complete the experiment, custom Network ACLs must be created. The process is initiated by navigating to the Network ACLs section under the VPC Security menu and selecting **Create network ACL**.

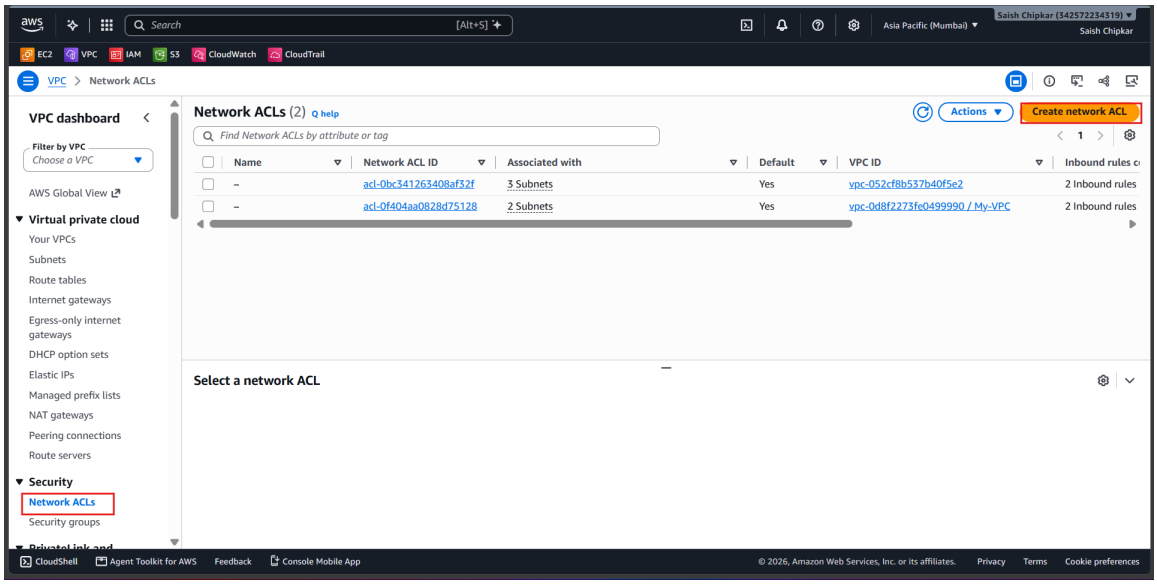


Figure 11: Initiating the creation of a custom Network ACL

4.1 Creating Custom Network ACLs

Two distinct Network ACLs were provisioned. The first, named **Public-NACL**, was created and assigned to the custom VPC.

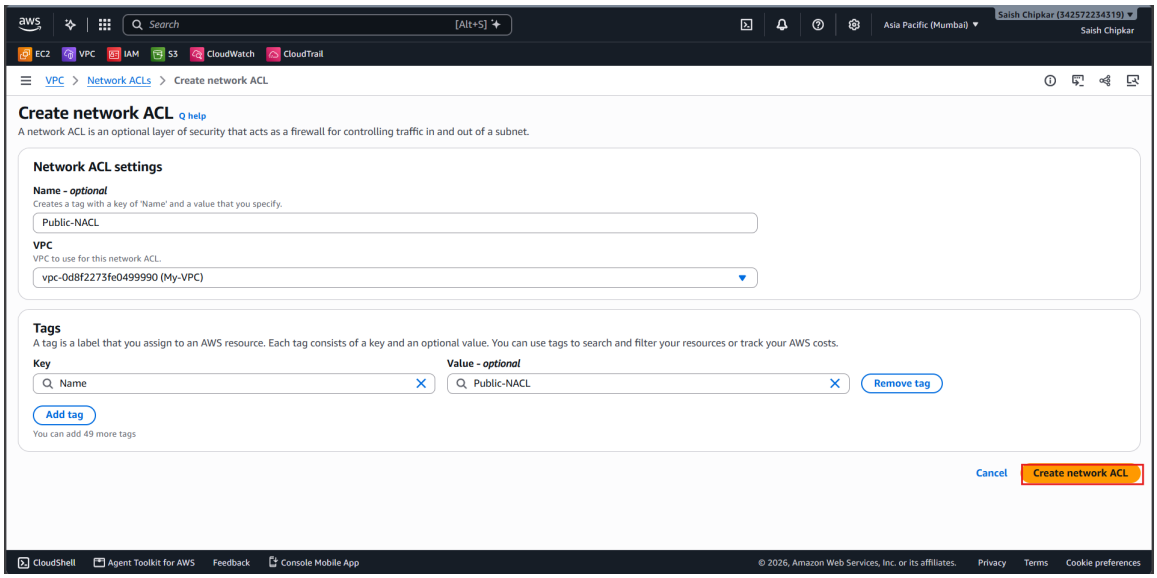


Figure 12: Creating the Public-NACL

A second Network ACL named **Private-NACL** was subsequently created to govern the traffic entering and leaving the isolated private subnet.

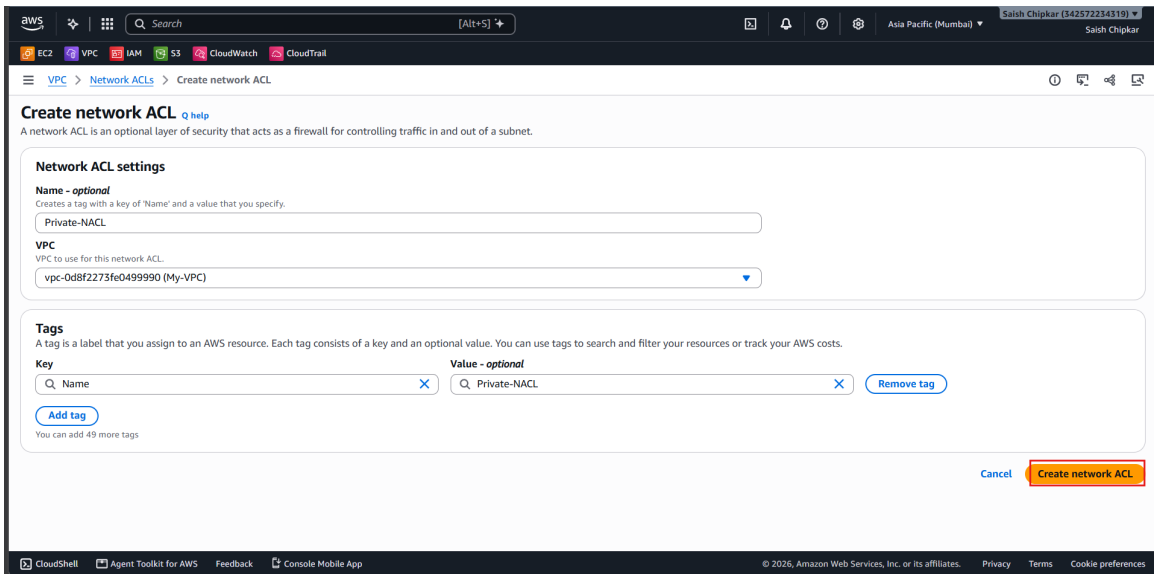


Figure 13: Creating the Private-NACL

4.2 Associating Subnets to Network ACLs

The newly created NACLs must be explicitly associated with their respective subnets. The **Public-NACL** was selected, and the "Edit subnet associations" option was accessed.

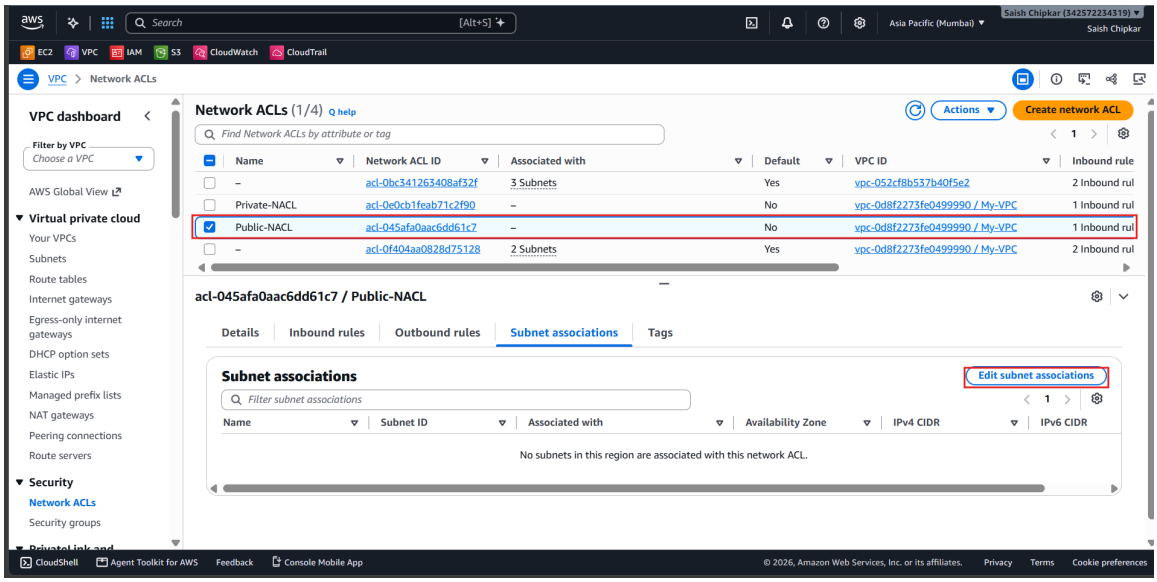


Figure 14: Initiating subnet association for Public-NACL

The **Public-Subnet** was selected from the list of available subnets, and the changes were saved, successfully binding this NACL to the public tier.

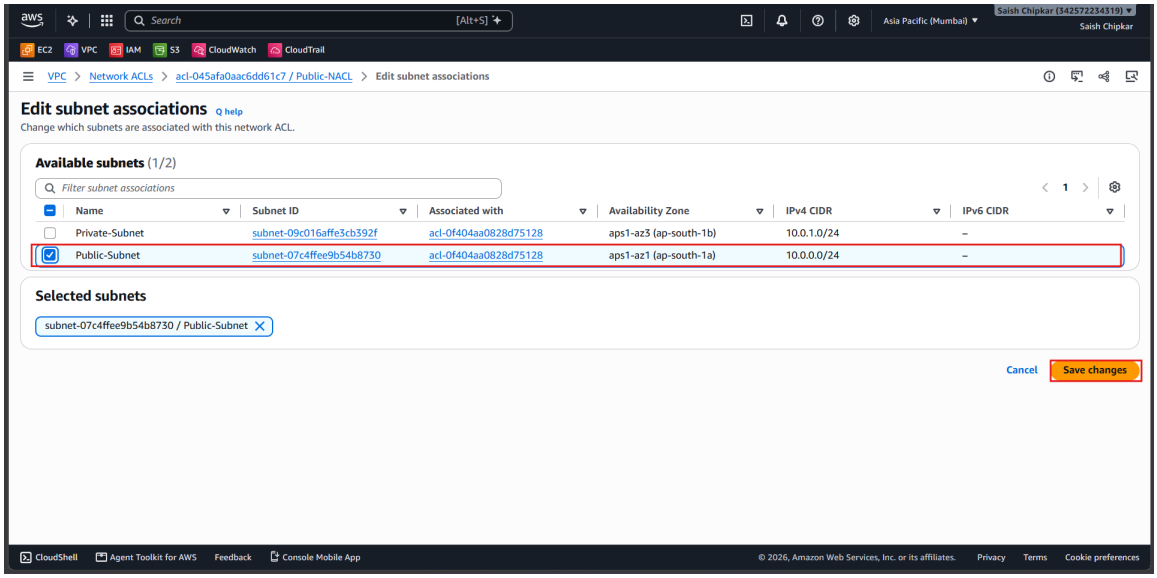


Figure 15: Associating Public-Subnet with Public-NACL

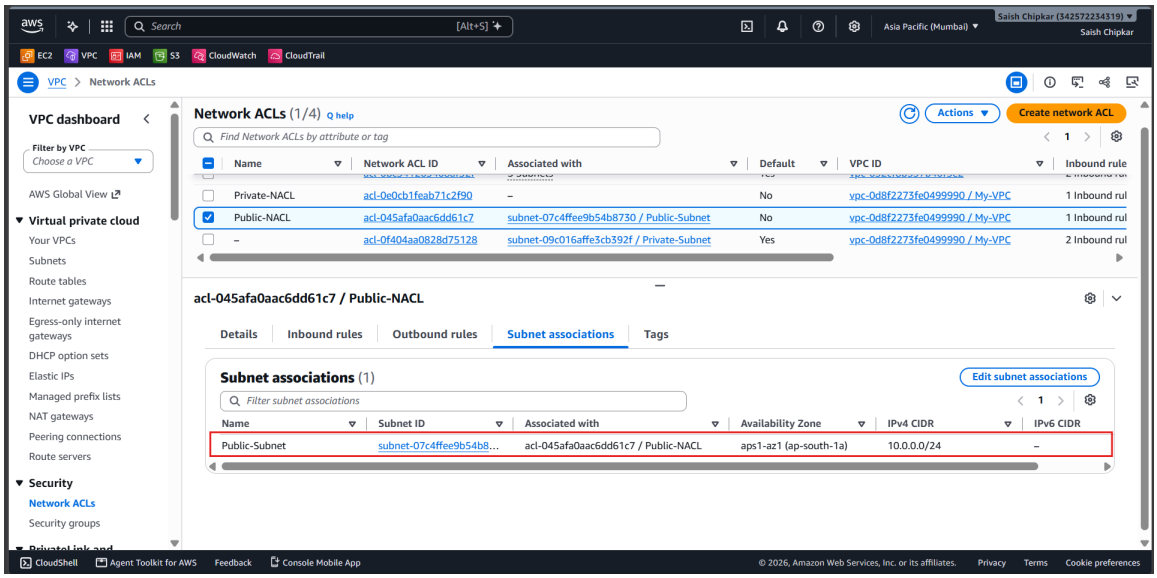


Figure 16: Confirmation of Public-Subnet association

Following the same procedure, the **Private-NACL** was selected to associate it with the internal tier.

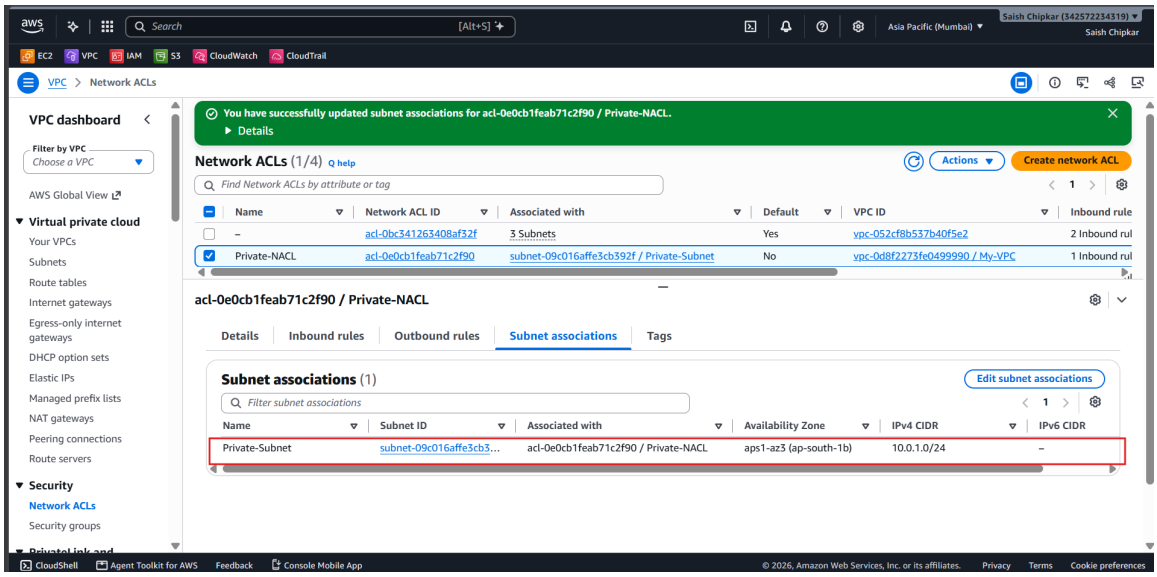


Figure 17: Initiating subnet association for Private-NACL

The **Private-Subnet** was selected and explicitly associated with the **Private-NACL**.

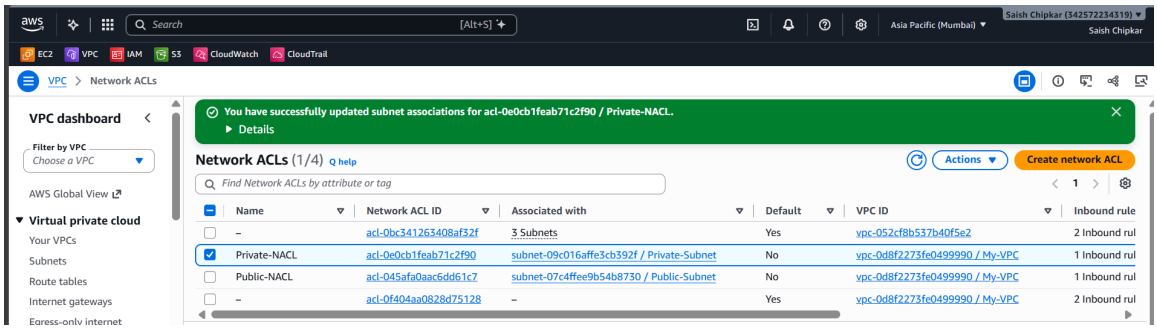


Figure 18: Associating Private-Subnet with Private-NACL

5 Configuring Security Group Rules

With the subnets bounded by specific NACLs, the next step involves defining the stateful firewall rules at the instance level via Security Groups.

5.1 Public Security Group (Public-SG) Rules

The **Public-SG** was selected from the Security Groups dashboard, and "Edit inbound rules" was clicked to define permissible incoming traffic to the public-facing server.

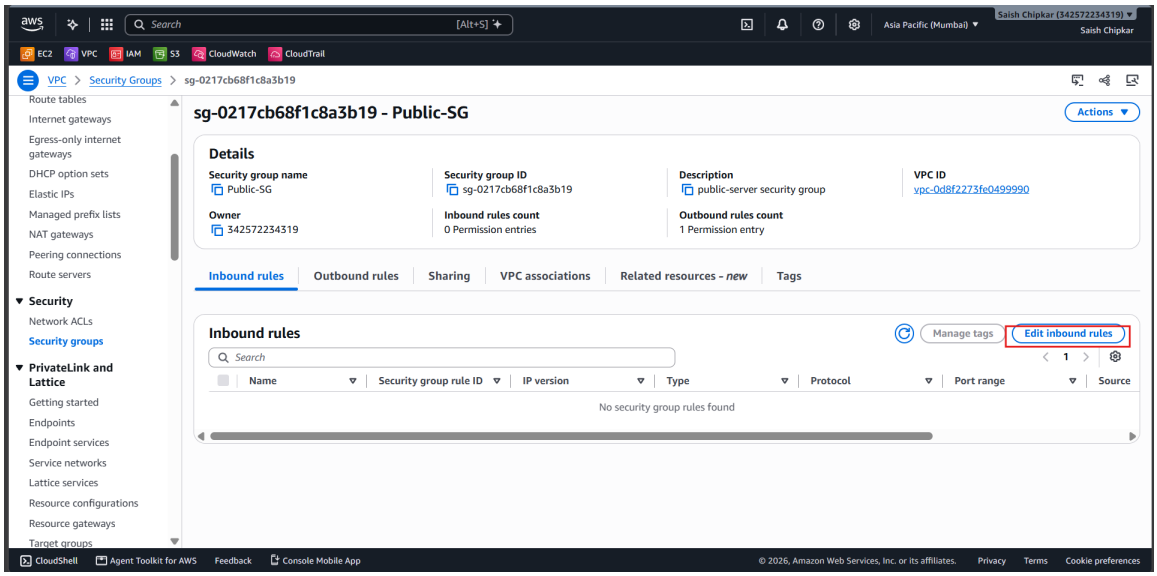


Figure 19: Editing inbound rules for Public-SG

To allow secure administrative access and connectivity testing, two inbound rules were added:

- **SSH (Port 22)**: Restricted exclusively to a specific custom IP address (`49.43.25.112/32`) for secure shell access.
- **All ICMP - IPv4**: Opened to `0.0.0.0/0` (Anywhere) to allow ping requests for network troubleshooting.

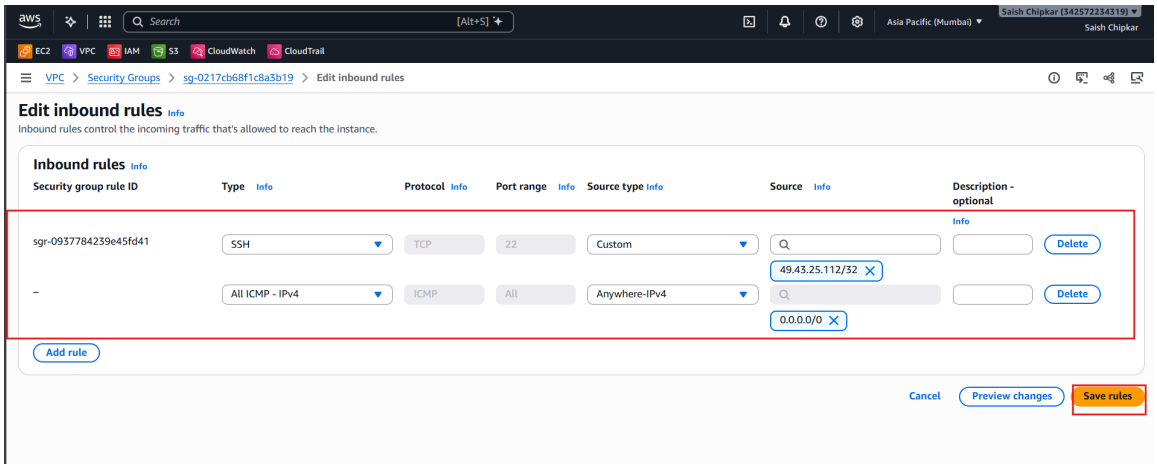


Figure 20: Adding SSH and ICMP inbound rules to Public-SG

5.2 Private Security Group (Private-SG) Rules

Next, the **Private-SG** was selected to begin configuring its inbound rules. This group will be heavily restricted, designed to only accept targeted internal traffic.

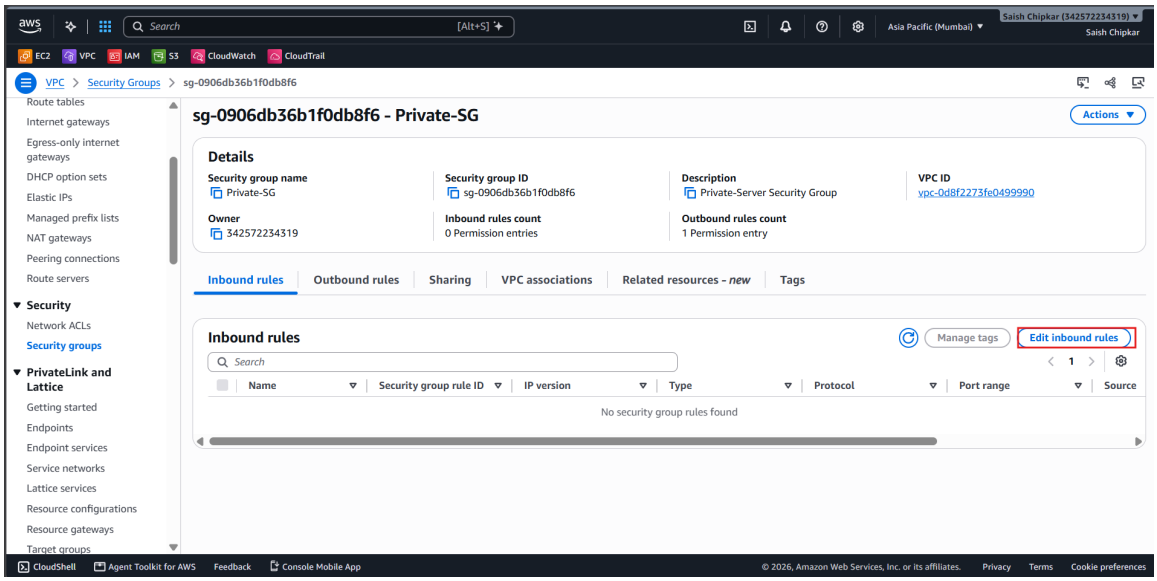


Figure 21: Initiating inbound rule configuration for Private-SG

To ensure complete isolation, a single rule was added to the **Private-SG** allowing only SSH (Port 22) traffic[cite: 22]. Crucially, the source was restricted to the Security Group ID of the **Public-SG** (`sg-0217...`)[cite: 22]. This ensures the private server can only be accessed via the public server acting as a bastion host.

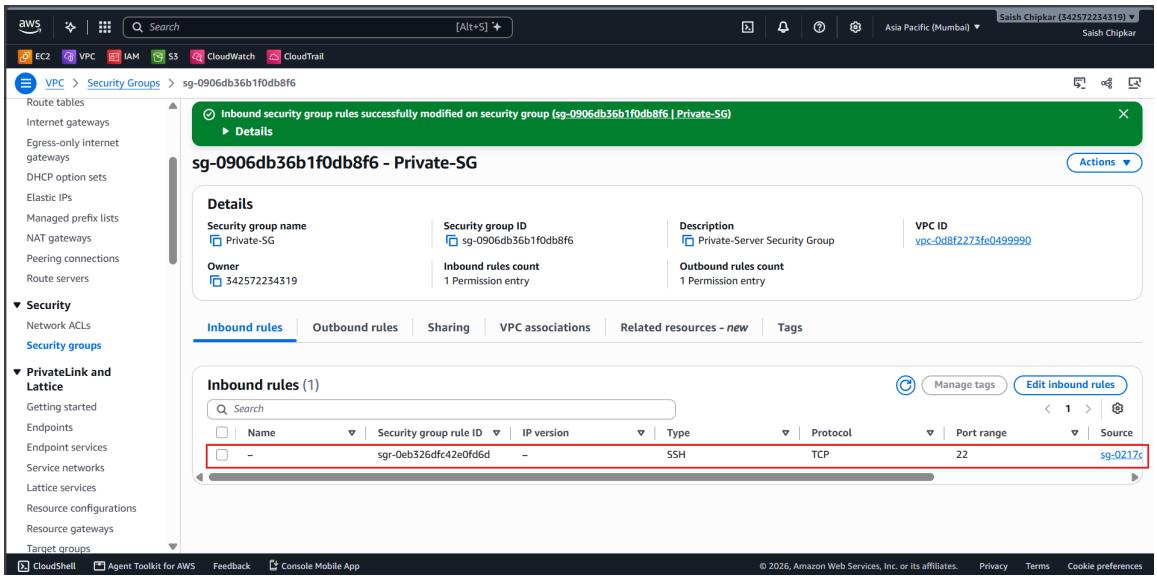


Figure 22: Restricting SSH access to Private-SG via Public-SG

6 Configuring Network ACL Rules

With Security Groups established, explicit rules must be defined within the custom Network ACLs to allow necessary traffic at the subnet boundary.

6.1 Public Network ACL (Public-NACL) Rules

The **Public-NACL** was selected, and the "Edit inbound rules" interface was opened[cite: 23].

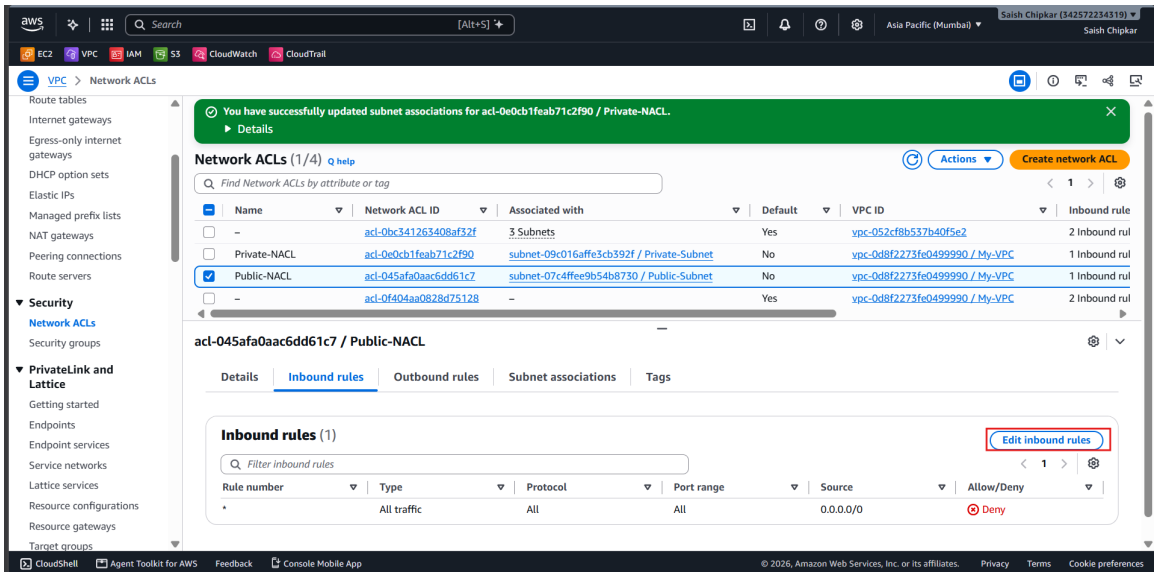


Figure 23: Initiating inbound rule editing for Public-NACL

Two explicit inbound rules were created to permit traffic into the public subnet[cite: 24]:

- **Rule 100 (SSH)**: Allows TCP port 22 traffic from `0.0.0.0/0` (Anywhere)[cite: 24].
- **Rule 101 (Custom ICMP - IPv4)**: Allows Echo Reply traffic specifically from the local VPC CIDR (`10.0.1.0/24`)[cite: 24].

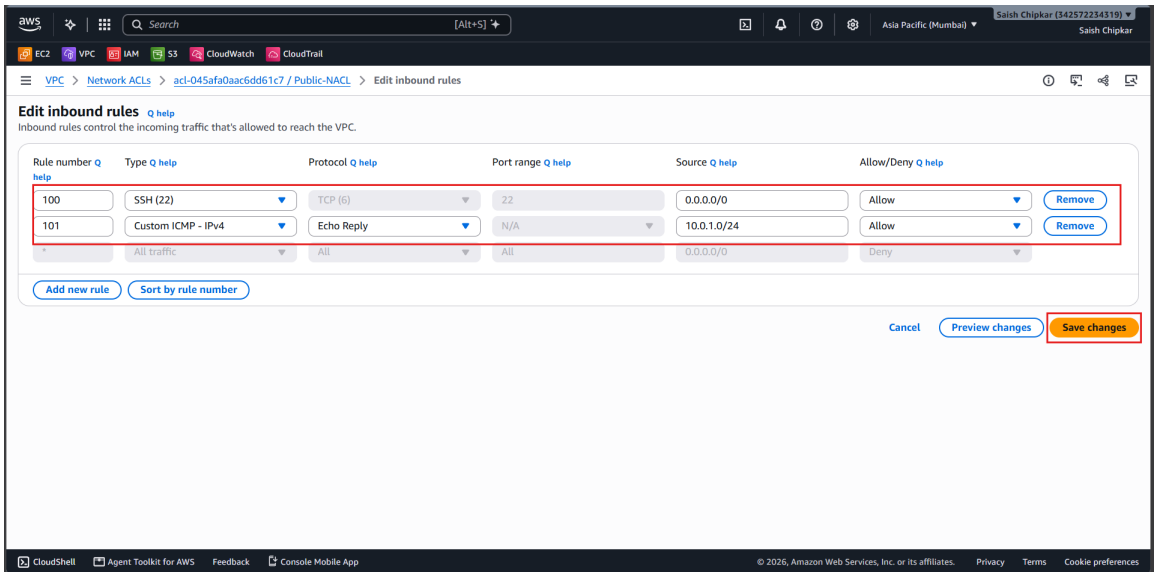


Figure 24: Defining inbound rules (SSH and ICMP) for Public-NACL

Next, outbound rules for the **Public-NACL** were configured[cite: 25].

- **Rule 99 (Custom TCP)**: Allows ephemeral port traffic (1024 - 65535) out to `0.0.0.0/0` to enable return traffic for established connections[cite: 25].
- **Rule 100 (Custom ICMP - IPv4)**: Allows Echo Request traffic out to the specific internal subnet (`10.0.1.0/24`)[cite: 25].

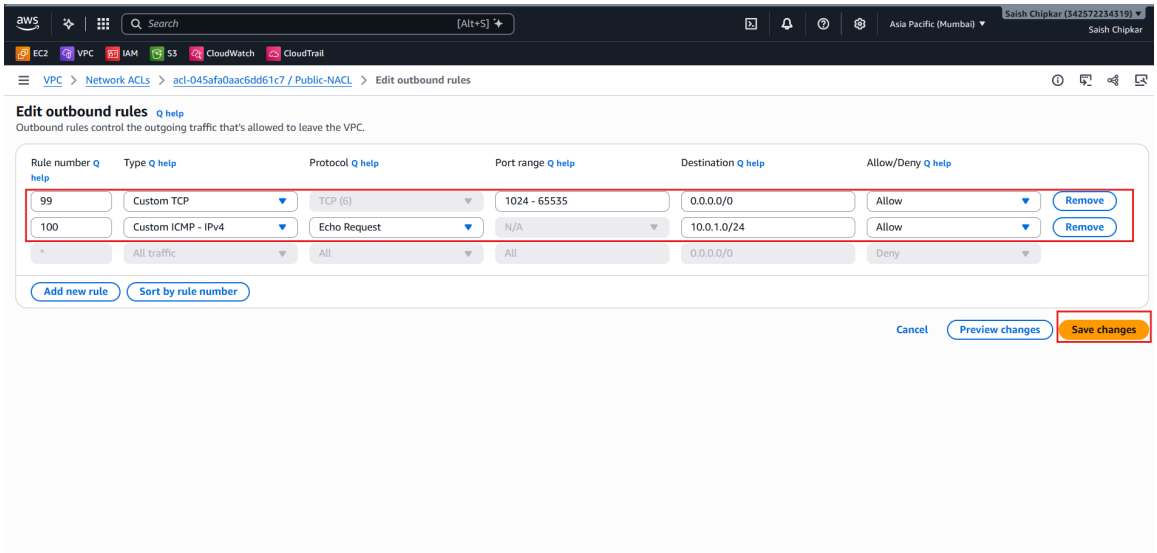


Figure 25: Defining outbound ephemeral port and ICMP rules for Public-NACL

6.2 Private Network ACL (Private-NACL) Rules

To govern traffic at the boundary of the private subnet, the **Private-NACL** inbound rules were configured[cite: 26].

- **Rule 100 (Custom ICMP - IPv4)**: Explicitly allows incoming Echo Request traffic originating from the public subnet CIDR (`10.0.0.0/24`)[cite: 26].

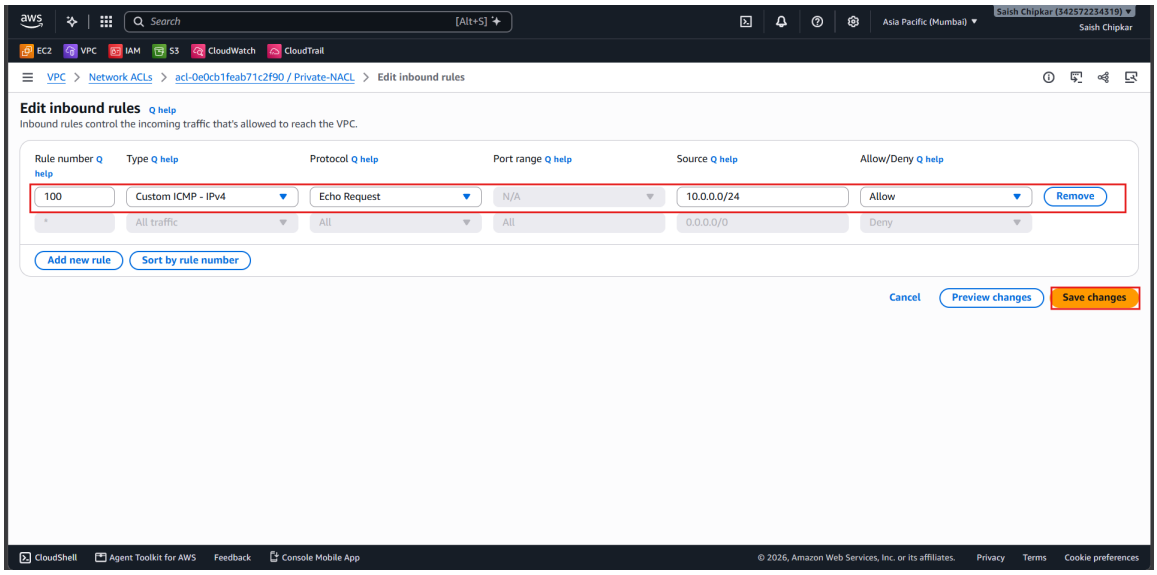


Figure 26: Defining inbound ICMP rule for Private-NACL

The corresponding outbound rule for the **Private-NACL** was then established[cite: 27].

- **Rule 100 (Custom ICMP - IPv4)**: Allows outgoing Echo Reply traffic destined for the public subnet CIDR (`10.0.0.0/24`)[cite: 27].

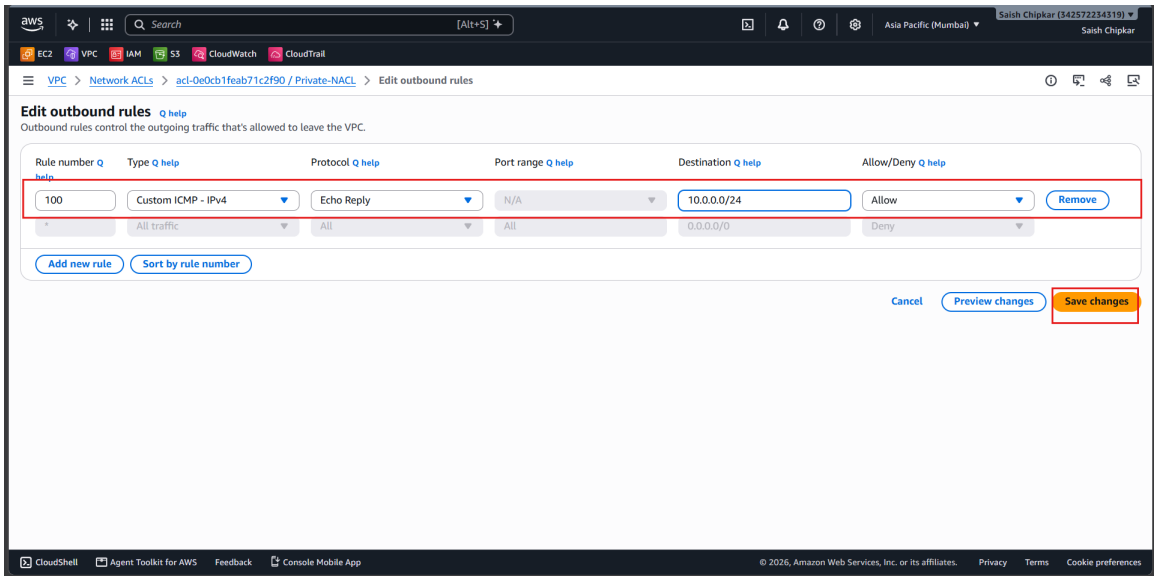


Figure 27: Defining outbound ICMP rule for Private-NACL

7 Route Table Configuration

The final networking step ensures traffic is routed correctly between the subnets and the internet.

7.1 Public Route Table

A route table designated **Public-RT** was configured to manage traffic for the public subnet[cite: 28, 29]. A default route (`0.0.0.0/0`) was added, targeting an Internet Gateway (`igw-...`)[cite:

28]. This route provides the public subnet with a path to the internet[cite: 28]. The **Public-Subnet** was then explicitly associated with this route table[cite: 29].

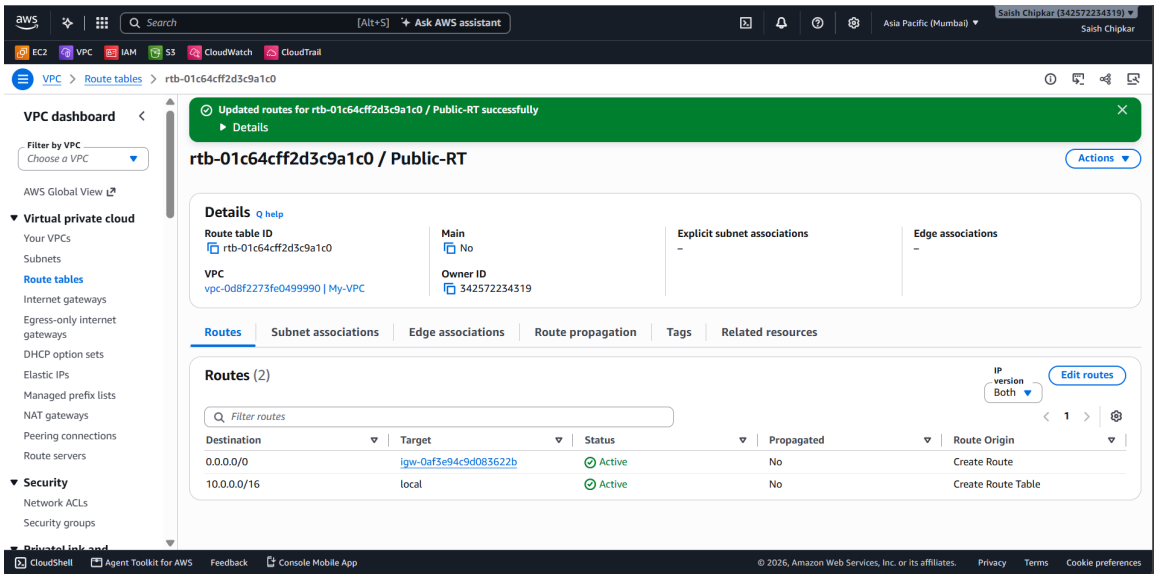


Figure 28: Public-RT configured with an Internet Gateway route

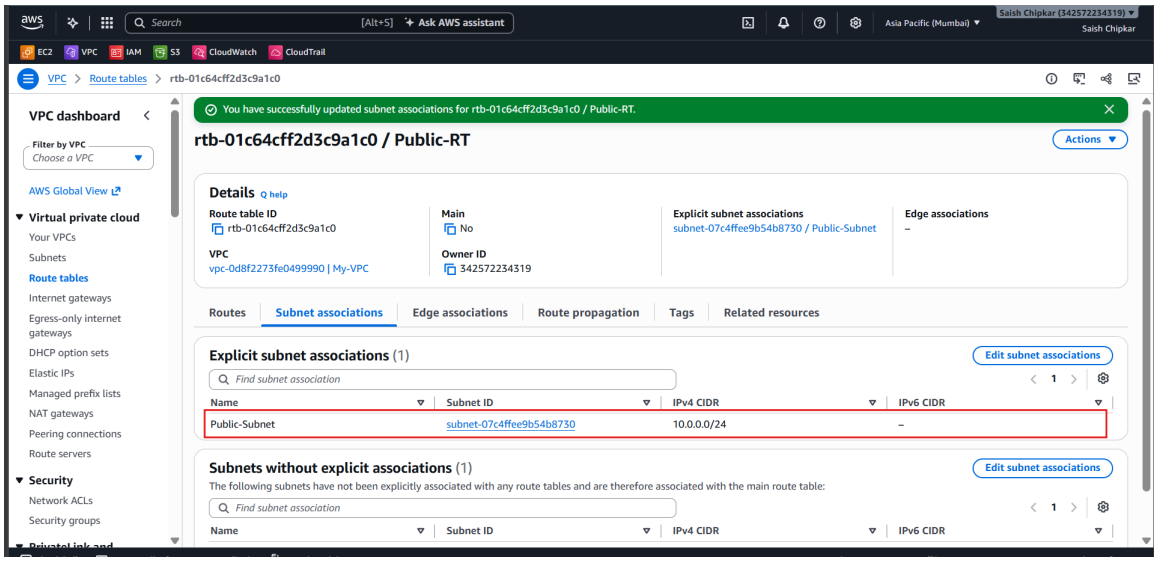


Figure 29: Explicit association of Public-Subnet to Public-RT

7.2 Private Route Table

A separate route table, **Private-RT**, was created for the internal subnet[cite: 30]. This table only contains the default local route (`10.0.0.0/16`), ensuring traffic remains entirely within the VPC and has no path to the internet[cite: 30].

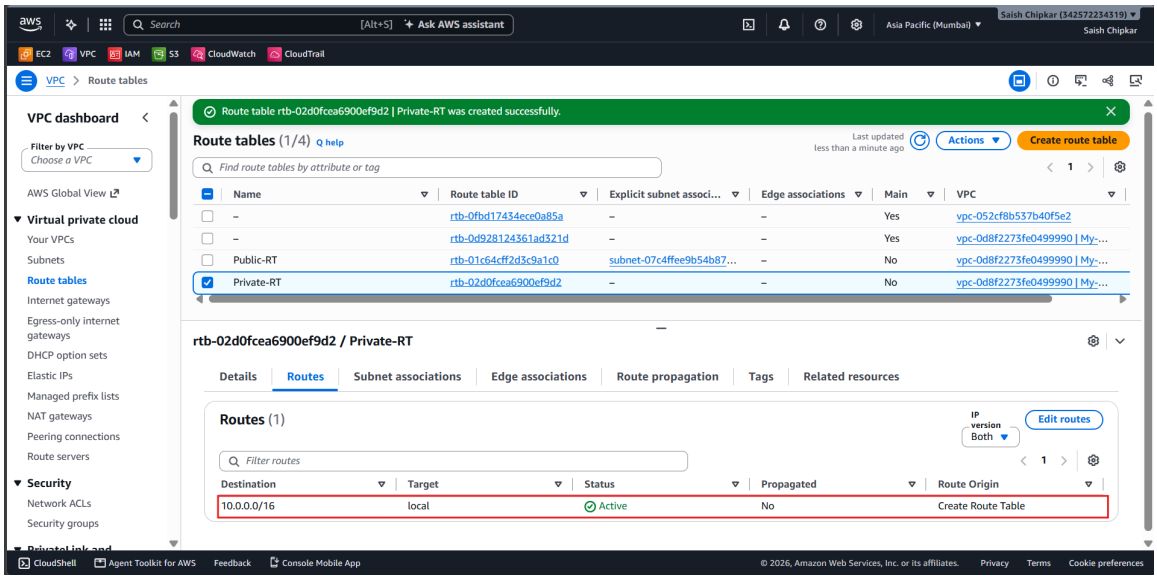


Figure 30: Private-RT displaying only the local VPC route

The **Private-Subnet** was then associated with this internal route table, completing the network isolation[cite: 31].

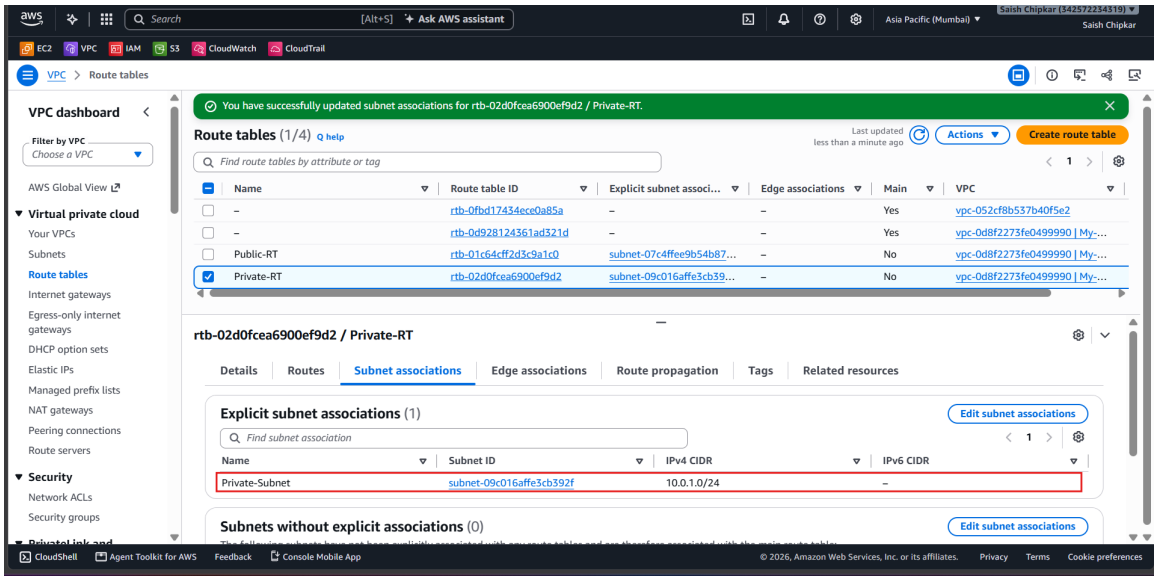


Figure 31: Explicit association of Private-Subnet to Private-RT

8 Experiment Verification

To confirm the architecture functions as intended, a connectivity test was performed from the public bastion host to the isolated private instance.

First, the **Private IPv4 address** of the **Private-Server** was identified from the EC2 Instances dashboard (`10.0.1.28`)[cite: 32].

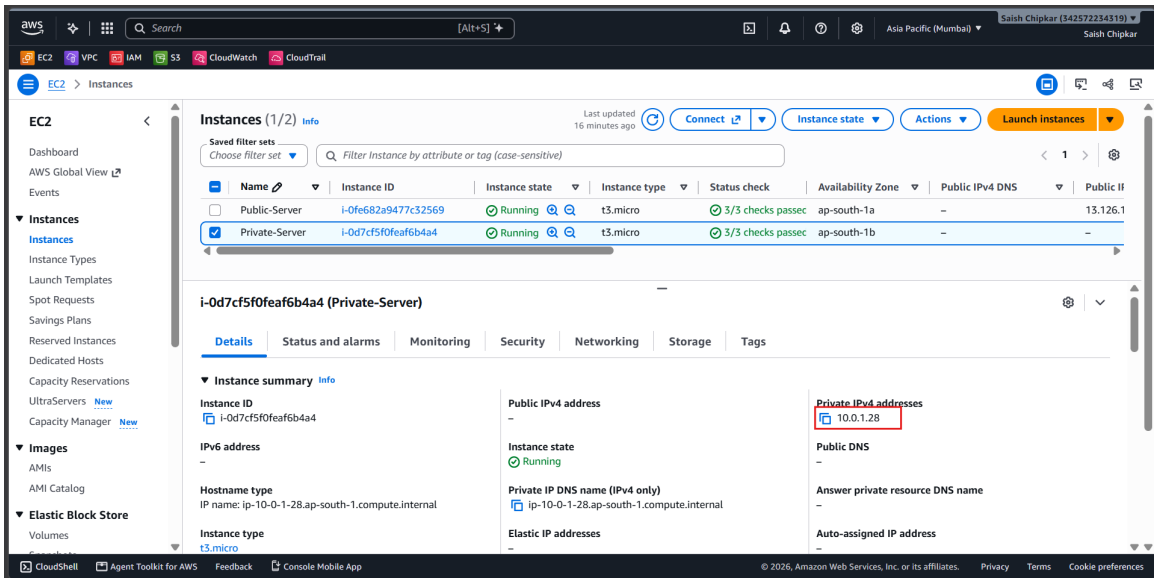


Figure 32: Locating the Private IP address of the Private-Server

Next, an SSH connection was established from the local terminal to the **Public-Server** using its public IP address (`13.126.143.235`) and the corresponding key pair (`key18.pem`)[cite: 33]. Once logged into the Amazon Linux 2023 environment on the Public-Server, a `ping` command was executed targeting the Private-Server's IP (`10.0.1.28`)[cite: 33].

The test was successful, demonstrating zero percent packet loss[cite: 33]. This confirms that the Security Group and Network ACL rules correctly permitted ICMP Echo Requests from the public subnet to enter the private subnet, and successfully allowed the Echo Replies to return to the public instance[cite: 33].

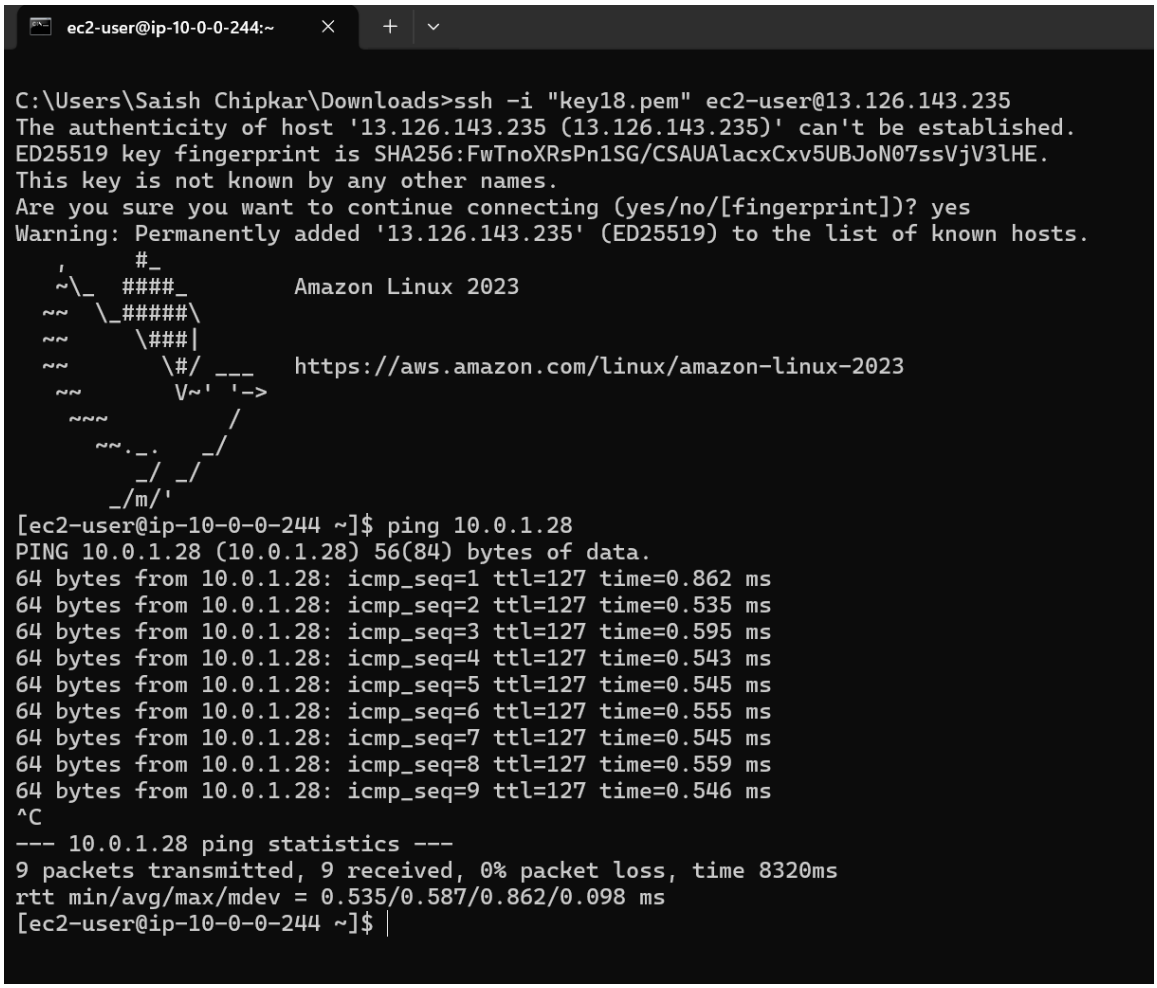


Figure 33: Successful SSH login to Public-Server and ping to Private-Server

9 Conclusion and Conceptual Summary

This experiment practically demonstrates the AWS "Defense in Depth" security model by layering two distinct but complementary network filtering mechanisms: Network Access Control Lists (NACLs) and Security Groups.

9.1 The Real-World Concept

In a real-world architectural analogy, a VPC is akin to a corporate office building. The **Subnets** represent distinct zones within that building—such as a public reception lobby (Public Subnet) and a highly restricted vault (Private Subnet).

- **Network ACLs (The Perimeter Guards):** NACLs act as the security checkpoints at the boundary of each floor or zone. They are *stateless*, meaning the guard has no memory of who just walked in; every single entry and exit must be evaluated against a strict, numbered list of rules.
- **Security Groups (The Office Door Locks):** Security Groups function as the personalized smart locks on individual office doors (the EC2 instances). They are *stateful*, meaning if the lock permits an authorized request to enter, it automatically remembers that request and allows the response to exit, without needing an explicit outbound rule.

9.2 Connection to the Experiment Activities

The activities performed in this experiment directly map to this dual-layered security concept:

1. **Stateless vs. Stateful Implementation:** When we configured the Security Groups, we only needed to define *inbound* rules (allowing SSH and ICMP). The stateful nature of the SG automatically allowed the replies back out. However, when configuring the NACLs, we had to explicitly define both *inbound* (Echo Request) and *outbound* (Echo Reply) rules, proving their stateless nature at the subnet boundary.

2. **Perimeter vs. Instance Control:** The Public-NACL allowed broad, general traffic (like SSH from anywhere, `0.0.0.0/0`) into the public subnet. However, the specific Public-SG on the EC2 instance acted as the final line of defense, narrowing that SSH access down to one single, specific administrator IP address (`49.43.25.112/32`).

3. **The Bastion Host Architecture:** By removing the Internet Gateway route from the Private-RT, we isolated the Private-Server. We then used Security Group referencing (allowing SSH on the Private-SG only if it originated from the Public-SG ID). This perfectly modeled a real-world **Bastion Host** (or Jump Server) pattern, where backend databases can only be accessed by first securely logging into a heavily fortified public-facing server.

Ultimately, this experiment confirms that while Security Groups offer fine-grained, instance-level protection, combining them with Network ACLs provides an immutable, overarching fall-back defense that protects the entire network tier even if an individual instance's firewall is compromised or misconfigured.